Auditing LLM-Generated GPU Kernel Claims with Contract-Aware Holdout Re-Evaluation

Aviad Cohen
Independent Researcher

Abstract

LLM systems generate multiple GPU kernels and often promote the fastest measured candidate, coupling search to evaluation. Timing noise, candidate multiplicity, or evaluator lifecycle errors can then become performance claims. We present OpenKernelForge, a contract-aware protocol that separates screening from independent confirmation using persistent modules, randomized paired CUDA-event blocks, fresh processes, formal controls, and a checksummed artifact ledger. On a performance-blind subset of 48 official KernelBench L1 tasks, 27 of 141 evaluated Gemini candidates passed the full gate, covering 10 tasks; none of the frozen task winners exceeded eager by the prespecified 2% margin in either screening or seven-process confirmation. One frozen candidate nevertheless confirmed $2.001\times$ versus the compiler baseline while remaining below eager. A deterministic fused8 grid first supplied a negative control: all 80 easy-regime candidates remained above the margin. We then calibrated a disjoint 32-candidate stress test near parity. At budget $K = 8$, three of four tasks appeared above the margin in screening, but only two confirmed; the median log optimism was 0.0072 with four-task bootstrap interval $[-0.0124, 0.0266]$. Thus the campaign observes one false promotion while not supporting a population effect. A lifecycle ablation separately measured $1.053\times$ synchronized-host inflation with no corresponding enclosing-event change. These results support independent confirmation as a useful default. We do not establish benchmark-wide performance.

1 Introduction

LLM-based systems can generate many plausible GPU kernels for one tensor program. The system then selects a candidate using measured latency. This makes evaluation part of the search algorithm: as the candidate budget grows, so does the chance of selecting a favorable timing fluctuation. Reusing that measurement as final evidence produces a systems analogue of test-set overfitting. A second risk comes from the evaluator itself. Incorrect state initialization, model construction inside timed regions, or permissive fallback paths can create a large apparent speedup without a better kernel.

Existing kernel benchmarks emphasize correctness and performance over practical baselines [Ouyang et al., 2025, Zhang et al., 2026, Lange et al., 2025], while recent harness work stresses constrained execution and artifact archival [Shui et al., 2026]. Systems measurement research has separately shown that small experimental choices can reverse performance conclusions [Mytkowicz et al., 2009, Curtsinger and Berger, 2013]. We connect these concerns through a promotion protocol for generated kernels. The protocol uses one dataset to screen candidates and fresh process-level data to confirm a frozen winner.

We ask three questions with separate evidence paths. **RQ1: repeatability.** Across a corrected KernelBench subset, how many valid tasks, screening wins, and independently confirmed wins remain under a fixed three-candidate budget? **RQ2: selection bias.** Across an easy deterministic grid and a calibrated near-threshold fused8 stress test, how do apparent and independently confirmed wins

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

change with candidate budget? **RQ3: evaluator validity.** In a separate disposable-process ablation, how much apparent speedup is created by reconstructing and transferring a reference model inside the measured call?

The contribution is an evaluation design rather than a generation model: a contract-valid persistent lifecycle with frozen provenance; screen-then-confirm timing across randomized paired blocks and fresh processes; fail-closed null, known-slowdown, and lifecycle controls; and an artifact ledger that makes both the candidate and evaluator auditable.

The claim ladder is deliberately strict:

$$\text{contract-valid and correct} \;\rightarrow\; \text{beats compiler} \;\rightarrow\; \text{beats eager} \;\rightarrow\; \text{independently confirmed.}$$

Each arrow is a separate empirical claim.

Recent verifier studies show that kernel correctness requires more than a single numerical comparison [Shah and Shrestha, 2026, Li et al., 2026]. Accordingly, our protocol treats correctness, output contracts, lifecycle validity, and performance as distinct gates. The historical fused8 example is motivating evidence only. The corrected campaign reported here passed its prespecified control gates and is derived from checksummed raw artifacts.

2 Contract-aware holdout confirmation

Lifecycle and correctness. Official KernelBench tasks define a reference `Model` and expect a persistent `ModelNew`. We instantiate both from the same frozen constructor arguments with restored RNG state and move them to the GPU before warmup. Source, initialization, and before/after state hashes preserve provenance. Candidates must pass static policy, five seeded input generations, repeated execution, output-tree, alias, input-effect, special-value, shape, dtype, and official tolerance checks. A pre-timing `TorchDispatchMode` audit rejects high-level ATen compute and requires an observed Triton launch. These checks are contract guardrails, not a security sandbox.

Performance-blind selection. The study uses official KernelBench L1 commit `423217d9`. Before candidate generation, memory preflight selected 48 tasks by deterministic family-round-robin with lexical within-family order and an 8 GiB known-residency cap. The original target of 50 was amended before manifest freeze: 49 of 100 tasks met the unchanged cap, the next tier began at 20 GiB, and one feasible task was reserved for the excluded-task shakedown. No candidate or timing field informed the amendment.

Screening and confirmation. For task t, candidate c, process r, and paired block b, define

$$z_{tcrb} = \log(\widetilde{T}^{\text{eager}}_{trb} / \widetilde{T}^{c}_{tcrb}), \tag{1}$$

where each $\widetilde{T}$ is per-launch CUDA-event latency on the same input snapshot. Twenty randomized paired blocks screen three candidates per task in one process. We freeze the candidate with the largest median z; tasks with no valid candidate remain explicit INVALID rows. The winner then receives 20 blocks in each of seven fresh processes with new seeds. Processes are split into separately invoked waves of four and three, separated by at least 30 minutes on the same GPU and software fingerprint. Each interval adapts its launch count to at least 1.5 ms; method order is randomized and cache state is perturbed identically with a read-write buffer sized from reported L2 capacity. Compile cost is separated from runtime, and promotion is paired against eager.

The practical margin is $\delta = 0.02$. This fixed protocol default was chosen before timing to require a nontrivial practical gain rather than parity; it was not calibrated post hoc to the observed T4 noise. Primary outcomes are valid-task, screening-win, and confirmed-win counts; false-promotion fraction; and median screening-to-confirmation optimism with a task-bootstrap interval. A secondary process-cluster analysis uses a one-sided lower bound, centered bootstrap test, and Benjamini–Hochberg correction. Exact seeds, resampling rules, telemetry, precision settings, and failure behavior are fixed in the executable protocol and summarized in the appendix.

Controls and multiplicity. Seven-process calibration compares eager with an identical wrapper and a calibrated extra-work wrapper. A separate 24-process lifecycle ablation measures reference

Table 1: Completed corrected campaign. External winners remain below eager; a near-threshold stress test exposes one screen-only promotion; lifecycle timing isolates host-side inflation.

Study	Evidence units	Screen	Confirm	False prom.	Key estimate
RQ1 KernelBench	10/48 valid	0	0	–	1.014 ratio $[0.915, 1.170]$
RQ2 easy, $K = 20$	4/4 tasks	1.00	1.00	0	0.00000 log optimism
RQ2 near, $K = 8$	4/4 tasks	0.75	0.50	1/3	0.00725 log optimism
RQ3 lifecycle	24/24 rows	–	–	–	1.053 host $[0.998, 1.114]$

reconstruction and transfer with synchronized host latency and an enclosing CUDA-event diagnostic. Missing or failed controls block performance claims. RQ2 uses two deterministic fused8 regimes. An easy grid freezes and confirms 20 candidates per task for $K \leq 20$. A separate stress test freezes 20 delayed variants per task, uses three disjoint calibration processes to select eight variants within a predeclared $[0.98, 1.04]$ speedup window, then excludes calibration and screens and confirms all 32 selected candidates. Its delayed variants preserve the base output while copying the first input into unused scratch with calibrated Triton passes; fractional work units copy a prefix. Its seven confirmation processes begin at least 30 minutes after screening, and 4,000 resamples estimate budgets $K \in \{1, 2, 3, 5, 8\}$. The two regimes use separate artifacts and GPUs and are not inferred from the KernelBench pool.

3 Corrected campaign results

The historical fused8 artifacts provide one motivating reversal: a deterministic `bias_relu` template measured 1.029× above eager in one screening run but 0.976× at repeated median. Model sampling cannot explain the change. We do not treat this case as a population frequency. The corrected campaign instead separates three questions and reports complete, checksummed evidence for each (Table 1 and Figure 1).

RQ1: validity precedes speed. All 144 Gemini candidates were frozen before timing. Of 141 evaluated records, 77 failed static policy, none failed only an output contract, 9 failed numerical or repeat correctness, 28 failed during candidate compilation or execution, and 27 passed the full gate. A separate compiler-baseline OOM blocked the remaining task's three candidates before evaluation. The valid candidates covered 10 of 48 tasks, all matrix multiplications. One winner beat compiler (1.826×) but not eager (0.937×); a separate seven-process A4500 check confirmed 2.001× versus compile. Every frozen winner was below eager in screening and confirmation, so the false-promotion fraction is undefined rather than zero. Across the 10 valid tasks, the median screening-to-confirmation ratio was 1.014 with task-bootstrap interval $[0.915, 1.170]$. The broad interval contains parity and does not support a directional selection-optimism claim.

RQ2: multiplicity depends on the performance regime. The easy grid remains a negative control: all 80 deterministic candidates confirmed above the margin, so apparent and confirmed win rates were 1.0 for every $K \leq 20$ on the original T4 and a same-GPU A4500 replication. The disjoint calibrated stress test instead placed 32 frozen candidates near parity. At $K = 8$, screening promoted three of four task winners, whereas independent confirmation retained two. The removed `bias_gelu` winner moved from 1.0271× to 1.0001×; `bias_relu` and `rmsnorm` remained above the margin. Across candidate resamples, apparent and confirmed win rates diverged from 0.154 versus 0.124 at $K = 1$ to 0.750 versus 0.500 at $K = 8$ (Figure 1). Median log optimism at $K = 8$ was 0.00725, but its four-task bootstrap interval $[-0.01239, 0.02661]$ contains zero. The observed false promotion therefore supports the protocol's mechanism without establishing a population rate. Across frozen full-budget winners, RQ1 remains at zero wins for margins from 0–5%; the near-threshold screen-only promotion persists at 1%, 2%, and 3% (Appendix A).

RQ3: timing boundaries matter. The lifecycle control completed 24/24 process rows across eight tasks. Rebuilding and transferring the reference inside the measured call changed synchronized host latency by a median factor of 1.053, while the median enclosing CUDA-event ratio was 1.0001. Across the 24 process medians, the respective IQRs were $[0.998, 1.114]$ and $[0.9919, 1.0019]$. The

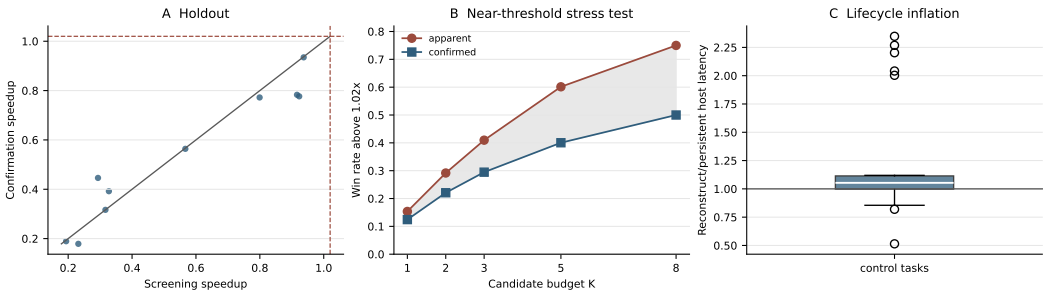

Figure 1: Corrected campaign outcomes. Panel A shows the 10 valid KernelBench winners, all below the 1.02× margin in screening and fresh-process confirmation. Panel B shows the calibrated stress test's apparent and independently confirmed win rates; their gap grows with candidate budget. Panel C shows reconstruct-per-call host-lifecycle inflation across 24 control processes.

difference shows that host lifecycle contamination can alter end-to-end claims without changing device execution, and that the timing boundary must be stated explicitly.

4 Positioning, limitations, and conclusion

KernelBench measures generated-kernel capability at scale [Ouyang et al., 2025]; KernelBench-Verified adds hidden distributions, realistic precision settings, and memory analysis [Zhang et al., 2026]; robust-kbench broadens verification scenarios [Lange et al., 2025]; and FastKernels studies production alignment [Oliaro et al., 2026]. Harness Engineering archives workload-grounded optimization evidence [Shui et al., 2026]. OpenKernelForge does not compete on generation capability. Its narrower focus is the statistical and lifecycle validity of promoting one measured winner after candidate search. Contract-Grade Verifier formalizes structural and behavioral kernel contracts [Shah and Shrestha, 2026], while Correct but Slow shows that weak verification can admit semantically deficient kernels that appear fast [Li et al., 2026]. Our verifier adopts the directly enforceable output-tree, alias, mutation, special-value, and repeated-execution checks, while leaving task-specific distribution and precision oracles explicit rather than claiming generic coverage.

The external study is limited to one Tesla T4, a deterministic feasible subset rather than a random sample of KernelBench L1, one model, and three candidates per task. The fused8 studies cover four tasks in one shape regime. The near-threshold stress test and an easy-grid hardware control ran on one RTX A4500. Its four-task bootstrap interval is wide, and calibration intentionally enriches near-threshold candidates rather than estimating their natural frequency. The 38/48 invalid-task count and the fact that every valid task is matrix multiplication show that candidate validity and family structure dominate this campaign. Seven processes across two time periods provide a practical holdout check but not precise tail modeling. The fixed 2% promotion margin is a prespecified practical threshold, not a noise-calibrated equivalence bound. Candidate generation is model-specific, whereas both multiplicity regimes use deterministic templates; the studies answer different questions. CUDA graphs, multi-GPU replication, deployment-cost amortization, and benchmark-wide model comparisons are outside scope. Compiler-relative evidence is also limited: the baseline was available for 47/48 selected tasks and all 10 valid-task winners at screening, while only one frozen winner received separate fresh-process compiler confirmation.

Generated-kernel systems need more than plausible source and one fast timing sample. The completed protocol makes screening, confirmation, and evaluator validity separately auditable. Here it changes the defensible conclusion: the corrected external campaign supports no above-eager kernel claim, the easy grid bounds multiplicity concerns, the calibrated stress test exposes a screen-only promotion, and the lifecycle ablation isolates a host-timing confound. Independent re-evaluation is therefore a useful default for generated-kernel claims, even when its result is a bound or a null finding rather than a new speedup.

References

Charlie Curtsinger and Emery D. Berger. STABILIZER: Statistically sound performance evaluation. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '13, pages 219–228. ACM, 2013. doi: 10.1145/2451116.2451141.

Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. Towards robust agentic CUDA kernel benchmarking, verification, and optimization, 2025. arXiv:2509.14279.

Tingxi Li, Ravishka Rathnasuriya, and Wei Yang. Correct but slow: An empirical study of the GPU kernel evaluation gap in modern domain-specific languages, 2026. arXiv:2607.04454.

Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '09, pages 265–276. ACM, 2009. doi: 10.1145/1508244.1508275.

Gabriele Oliaro, Yichao Fu, May Jiang, Owen Lu, Junli Wang, Zhihao Jia, Hao Zhang, and Samyam Rajbhandari. FastKernels: Benchmarking GPU kernel generation in production, 2026. arXiv:2605.23215.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels?, 2025. arXiv:2502.10517.

Rishi Shah and Rishav Shrestha. A contract-grade verifier for LLM-generated GPU kernels, and a native blackwell backward for the gated-linear-recurrence family, 2026. arXiv:2608.12700.

Yue Shui, Chenyu Ma, Hangfei Xu, Shengzhao Wen, and Yanpeng Wang. Harness engineering for LLM-driven GPU kernel generation, 2026. arXiv:2607.17979.

Yunxiang Zhang, Ping Yu, Jianyu Wang, Max (Xiangjun) Fan, Julian Reed, Azalia Mirhoseini, and Will Su. KernelBench-Verified: Do LLM-generated kernels actually beat PyTorch?, 2026. arXiv:2607.16241.

A Prespecified artifact contract

The executable protocol is stored in `configs/workshop2026_holdout_protocol.yaml`. Task selection writes a checksum-frozen JSON manifest, a compact CSV, and a SHA-256 digest before candidate generation. Candidate manifests bind each source file to the frozen task manifest, exact prompt and response hashes, configured model string, and the provider-returned model field when available. Screening freezes one winner per task in a second checksummed manifest before confirmation begins.

Each worker artifact contains policy and verification records, process identity, environment metadata, adaptive launch counts, cache-buffer size, randomized method order, input-snapshot hash, per-block CUDA-event latency, and clock/power snapshots. The analysis script consumes only those records and writes the screening winner set, aggregate promotion summary, process-cluster intervals, corrected p-values, promotion labels, and selection-optimism estimates.

Control gate and correctness matrix. Seven fresh calibration processes compare persistent eager with an identical wrapper and with a semantically identical wrapper containing calibrated extra GPU work. The gate requires the null median ratio in $[0.995, 1.005]$, no null promotion beyond the 2% practical margin, and detected slowdown in $[0.02, 0.08]$. A separate lifecycle ablation covers the first selected task per family, capped at eight, and requires three complete disposable processes per task, ten blocks per process, synchronized host end-to-end timing, and an enclosing CUDA-event diagnostic. Failure or absence of any required control blocks screening and invalidates affected claims.

Across the 24 lifecycle process rows, synchronized-host inflation had median 1.053 and IQR $[0.998, 1.114]$; enclosing-event inflation had median 1.0001 and IQR $[0.9919, 1.0019]$. A 20,000-sample task-cluster bootstrap, resampling the eight tasks while retaining their three process rows, gave intervals $[0.950, 1.557]$ and $[0.992, 1.212]$, respectively. These broad intervals expose task heterogeneity; the ablation establishes a timing-boundary mechanism rather than a population-wide effect size.

Correctness uses seeds 1103, 2207, 3301, 4409, and 5519. Every case checks two executions on the same logical input, exact nested output structure, shape, dtype, output-to-input and output-to-output alias patterns, input effects, NaN/+Inf/-Inf masks, and official task tolerances. The runtime policy then requires an observed Triton launch and rejects high-level ATen compute outside the allocation allowlist. These checks do not substitute for task-specific distribution, layout, or high-precision accumulation oracles.

Controlled multiplicity artifact contract. The easy RQ2 manifest freezes 20 deterministic template sources and metadata files for each of `bias_relu`, `residual_add_relu`, `bias_gelu`, and `rmsnorm_small` before timing. The near-threshold manifest separately freezes 20 delayed variants per task. Each adds a fixed number of Triton copy passes from the first tensor input into an otherwise unused scratch allocation, preserving the base output; fractional work units copy an input prefix. Three calibration processes select eight candidates per task inside the prespecified $[0.98, 1.04]$ window; calibration is excluded from primary estimates, and all selected candidates receive one screening and seven confirmation processes. Two earlier grid-design calibrations did not advance to primary screening and remain disclosed as design provenance. Reports preserve candidate counts, apparent and confirmed rates, per-task winners, and median selection optimism. Neither curve is inferred from the three-candidate KernelBench study.

Promotion-margin sensitivity. The 2% margin remains the prespecified primary threshold. A post-hoc sensitivity check over 0%, 1%, 2%, 3%, and 5% changes no RQ1 conclusion: all ten valid KernelBench winners remain below eager at every threshold. For the four frozen near-threshold winners, one screen-only promotion appears at 1%, 2%, and 3%; all four clear parity in both stages, while none clears 5%.

Historical audit boundary. The 17-page OpenKernelForge report remains an evaluator audit and implementation record. Historical KernelBench candidates and profiler rows are retained there for provenance. They are excluded from the corrected paper's performance tables. The internal fused8 summary remains usable only for the fixed observations whose artifacts survived; the full 160-candidate screening-to-confirmation frequency is not reconstructed from aggregate summaries.

Candidate funnel and compiler rung. The 144 generated candidates partition into 77 static-policy failures, zero contract-only failures, 9 numerical or repeat-correctness failures, 28 candidate compile/runtime failures, 27 full-gate passes, and 3 candidates not evaluated because their task's compiler baseline exhausted memory. These categories are mutually exclusive. Compiler baselines materialized for 47/48 selected tasks. Among the 10 frozen valid-task winners, 1 exceeded compiler parity and the 2% compiler margin at screening; that candidate was 1.826× versus compiler but 0.937× versus eager. A separate seven-process RTX A4500 check reused the frozen source and confirmed a 2.001× median candidate-versus-compile ratio; process medians ranged from 1.993× to 2.003×. Runtime excludes compile cost, while compile-and-first-call latency is preserved.

Completed artifact set. The corrected holdout contains 48 screening tasks, 70 winner-confirmation process records, and 249 checksum-ledger entries. The easy multiplicity study contains four screening records, 28 all-candidate confirmation processes, and a 67-entry ledger. The accepted near-threshold campaign contains 12 calibration, 4 screening, and 28 confirmation process records; its 94-file ledger verifies without error. Calibration passed in seven processes with null ratio 0.999998 and detected slowdown 0.0563. The lifecycle gate passed all 24 required process rows. The original holdout and easy grid used 2.091 recorded worker-hours on the Tesla T4; the accepted near-threshold v3 campaign used 0.368 worker-hours on the RTX A4500. A same-GPU A4500 replication of the easy grid added 32 complete worker records and 0.424 recorded worker-hours, preserving 1.0 apparent and confirmed rates at every budget. The compiler check added seven complete fresh-process records and 0.015 recorded worker-hours. The submission build rejects pending-evidence markers, checks the main matter against the strict four-page limit, and bundles the official, unmodified `neurips_2026.sty`.

Reviewer artifact. Source, executable protocols, derived tables, and release instructions are available at `https://github.com/aviad12g/kernel-forge`. The corresponding release attaches the checksummed campaign records used by this paper.